Assignment 6.1: Decision Trees and Random Forests

Name: [Your Name]
Course: [Course Number]: [Course Name]
Date: October 11, 2026

This notebook has two parts:

- **Activity 4.01:** Car data classification with a decision tree
- **Activity 4.02:** Random forest and extra trees (extreme random forest) classification

The goal is to help a car rental company pick cars that its clients are likely to buy. Each car is put into one of four classes: **unacceptable, acceptable, good, or very good**.

Part 1 – Activity 4.01: Car Data Classification

Step 1: Load the data

The car dataset is in the book's GitHub repository. I load it into a pandas DataFrame.

In [1]:
```
import pandas as pd

# link to the car dataset
file_url = 'https://raw.githubusercontent.com/PacktWorkshops/The-Applied-Artificial-Intelligence-Workshop/master/Datasets/car.csv'

# read the csv file into a DataFrame
df = pd.read_csv(file_url)
df.head()
```

Out[1]:

	buying	maintenance	doors	persons	luggage_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

Each row is one car. The columns are:

- **buying** – buying price

- **maintenance** – cost of maintenance
- **doors** – number of doors
- **persons** – how many people it can carry
- **luggage_boot** – size of the trunk
- **safety** – safety level
- **class** – how acceptable the car is (this is what we want to predict)

In [2]:
```
# number of rows and columns
print(df.shape)

# how many cars are in each class
df['class'].value_counts()
```

```
(1728, 7)
```

Out[2]:
```
class
unacc    1210
acc       384
good       69
vgood      65
Name: count, dtype: int64
```

Most of the cars are **unacc** (unacceptable). Only a few are **good** or **vgood**, so those classes will be harder to predict.

Step 2: Turn the text into numbers

All the values are text, like "vhigh" or "small". Scikit-learn needs numbers, so I use `LabelEncoder` to change each text value into a number.

In [3]:
```
from sklearn import preprocessing

# function that changes the text values in one column into numbers
def encode(data_frame, column):
    label_encoder = preprocessing.LabelEncoder()
    label_encoder.fit(data_frame[column].unique())
    return label_encoder.transform(data_frame[column])
```

In [4]:
```
# keep a copy of the class names before encoding
class_names = sorted(df['class'].unique())
print("Class numbers:", dict(enumerate(class_names)))

# encode every column
for column in df.columns:
    df[column] = encode(df, column)

df.head()
```

```
Class numbers: {0: 'acc', 1: 'good', 2: 'unacc', 3: 'vgood'}
```

Out[4]:

	buying	maintenance	doors	persons	luggage_boot	safety	class
0	3	3	0	0	2	1	2
1	3	3	0	0	2	2	2
2	3	3	0	0	2	0	2
3	3	3	0	0	1	1	2
4	3	3	0	0	1	2	2

After encoding, the classes are: **0 = acc, 1 = good, 2 = unacc, 3 = vgood**.

Step 3: Split the features and the label

`class` is the label. The other six columns are the features. Then I split the data so 90% is used for training and 10% for testing.

In [5]:
```
from sklearn import model_selection

# take out the class column as the label
label = df.pop('class')

# 90% training data, 10% testing data
features_train, features_test, label_train, label_test =
model_selection.train_test_split(
    df, label, test_size=0.1, random_state=88)

print("Training rows:", len(features_train))
print("Testing rows:", len(features_test))
```

```
Training rows: 1555
Testing rows: 173
```

Step 4: Build the decision tree

A decision tree asks a series of yes/no questions about the features (for example, "is safety low?") until it reaches a class.

In [6]:
```
from sklearn.tree import DecisionTreeClassifier

# create and train the decision tree
decision_tree = DecisionTreeClassifier(random_state=88)
decision_tree.fit(features_train, label_train)
```

Out[6]:

▾ DecisionTreeClassifier ⓘ ⓘ

▸ Parameters

▸ Fitted attributes

Step 5: Check how good the model is

In [7]:
```
# accuracy on the test data
dt_score = decision_tree.score(features_test, label_test)
print("Decision tree accuracy:", round(dt_score, 4))
```

```
Decision tree accuracy: 0.9538
```

In [8]:
```
from sklearn.metrics import classification_report, confusion_matrix

# precision, recall and f1-score for each class
dt_preds_test = decision_tree.predict(features_test)
print(classification_report(label_test, dt_preds_test,
target_names=class_names, zero_division=0))
```

```
              precision    recall  f1-score   support

         acc       0.87      0.98      0.92        42
        good       0.89      0.89      0.89         9
       unacc       0.99      0.96      0.98       114
       vgood       1.00      0.75      0.86         8

    accuracy                           0.95       173
   macro avg       0.94      0.89      0.91       173
weighted avg       0.96      0.95      0.95       173
```

In [9]:
```
# rows = real class, columns = predicted class
confusion_matrix(label_test, dt_preds_test)
```

Out[9]:
```
array([[ 41,   0,   1,   0],
       [  1,   8,   0,   0],
       [  3,   1, 110,   0],
       [  2,   0,   0,   6]])
```

What this shows: The decision tree gets about **95–96%** of the test cars right. It does very well on the big **unacc** class and also does well on the smaller classes. Only a few cars are put in the wrong class, which we can see from the small numbers outside the diagonal of the confusion matrix.

Part 2 – Activity 4.02: Random Forest Classification

The data is already loaded, encoded, and split from Part 1, so I use the same training and testing data here. That way the models can be compared fairly.

Step 1: Random forest

A random forest builds many decision trees on random parts of the data and lets them vote. I use 100 trees with a max depth of 6, like in the book.

In [10]:
```
from sklearn.ensemble import RandomForestClassifier

# 100 trees, each tree can be at most 6 levels deep
random_forest_classifier = RandomForestClassifier(n_estimators=100,
max_depth=6, random_state=168)
random_forest_classifier.fit(features_train, label_train)
```

Out[10]:

▾ RandomForestClassifier

▸ Parameters

▸ Fitted attributes

In [11]:
```
# predict the test data
rf_preds_test = random_forest_classifier.predict(features_test)

print("Random forest accuracy:",
round(random_forest_classifier.score(features_test, label_test), 4))
print(classification_report(label_test, rf_preds_test,
target_names=class_names, zero_division=0))
```

```
Random forest accuracy: 0.8439
              precision    recall  f1-score   support

         acc       0.67      0.76      0.71        42
        good       0.00      0.00      0.00         9
       unacc       0.92      0.96      0.94       114
       vgood       0.83      0.62      0.71         8

    accuracy                           0.84       173
   macro avg       0.60      0.59      0.59       173
weighted avg       0.80      0.84      0.82       173
```

In [12]:
```
confusion_matrix(label_test, rf_preds_test)
```

Out[12]:
```
array([[ 32,   0,  10,   0],
       [  8,   0,   0,   1],
       [  5,   0, 109,   0],
       [  3,   0,   0,   5]])
```

Step 2: Which features matter most?

The random forest can tell us how much each feature helped with the predictions.

In [13]:
```
# feature importance from the random forest
rf_varimp = pd.Series(random_forest_classifier.feature_importances_,
index=features_train.columns)
rf_varimp.sort_values(ascending=False)
```

Out[13]:
```
persons          0.352667
safety           0.336552
buying           0.126764
maintenance      0.103663
luggage_boot     0.059158
doors            0.021196
dtype: float64
```

persons and **safety** are by far the most important features. This makes sense, since a car that only fits 2 people or has low safety is almost always unacceptable. The number of **doors** barely matters.

Step 3: Extra trees (extreme random forest)

Extra trees works like a random forest, but it picks the split points at random instead of looking for the best one. This makes it faster and more random.

In [14]:
```
from sklearn.ensemble import ExtraTreesClassifier

# same settings as the random forest
extra_trees_classifier = ExtraTreesClassifier(n_estimators=100, max_depth=6,
random_state=168)
extra_trees_classifier.fit(features_train, label_train)
```

Out[14]:

▾ ExtraTreesClassifier ⓘ ?

▸ Parameters

▸ Fitted attributes

In [15]:
```
et_preds_test = extra_trees_classifier.predict(features_test)

print("Extra trees accuracy:",
round(extra_trees_classifier.score(features_test, label_test), 4))
print(classification_report(label_test, et_preds_test,
target_names=class_names, zero_division=0))
```

```
Extra trees accuracy: 0.815
              precision    recall  f1-score   support

         acc       0.61      0.67      0.64        42
        good       0.00      0.00      0.00         9
       unacc       0.89      0.98      0.93       114
       vgood       1.00      0.12      0.22         8

    accuracy                           0.82       173
   macro avg       0.62      0.44      0.45       173
weighted avg       0.78      0.82      0.78       173
```

In [16]:
```
confusion_matrix(label_test, et_preds_test)
```

Out[16]:
```
array([[ 28,   0,  14,   0],
       [  9,   0,   0,   0],
       [  2,   0, 112,   0],
       [  7,   0,   0,   1]])
```

In [17]:
```
# feature importance from extra trees
et_varimp = pd.Series(extra_trees_classifier.feature_importances_,
index=features_train.columns)
et_varimp.sort_values(ascending=False)
```

Out[17]:
```
safety          0.390638
persons         0.376620
buying          0.088445
maintenance     0.070233
luggage_boot    0.059659
doors           0.014404
dtype: float64
```

With a max depth of 6, both forests do worse than the single decision tree. They have trouble with the small classes. For example, they almost never predict **good**, because the trees are not deep enough to learn those rare cases.

Step 4: Optimize the models

To get better results for our clients, I try different values for `max_depth` and see which one gives the best accuracy on the test data.

In [18]:
```
# try different tree depths for both models
results = []
for depth in [4, 6, 8, 10, 12, None]:
    rf = RandomForestClassifier(n_estimators=100, max_depth=depth,
random_state=168)
    rf.fit(features_train, label_train)
    et = ExtraTreesClassifier(n_estimators=100, max_depth=depth,
random_state=168)
    et.fit(features_train, label_train)
    results.append({'max_depth': str(depth),
                    'random_forest': round(rf.score(features_test,
label_test), 4),
                    'extra_trees': round(et.score(features_test, label_test),
4)})

pd.DataFrame(results)
```

Out[18]:

	max_depth	random_forest	extra_trees
0	4	0.7688	0.6879
1	6	0.8439	0.8150
2	8	0.9249	0.8786
3	10	0.9538	0.9538
4	12	0.9711	0.9653
5	None	0.9711	0.9827

Deeper trees give much better results. Now I train the final models with no depth limit.

In [19]:

```
# final models with no depth limit
best_rf = RandomForestClassifier(n_estimators=100, max_depth=None,
random_state=168)
best_rf.fit(features_train, label_train)
best_rf_preds = best_rf.predict(features_test)

best_et = ExtraTreesClassifier(n_estimators=100, max_depth=None,
random_state=168)
best_et.fit(features_train, label_train)
best_et_preds = best_et.predict(features_test)

print("Optimized random forest")
print(classification_report(label_test, best_rf_preds,
target_names=class_names, zero_division=0))
print("Optimized extra trees")
print(classification_report(label_test, best_et_preds,
target_names=class_names, zero_division=0))
```

```
Optimized random forest
              precision    recall  f1-score   support

         acc       0.91      1.00      0.95        42
        good       1.00      0.89      0.94         9
       unacc       1.00      0.98      0.99       114
       vgood       0.86      0.75      0.80         8

    accuracy                           0.97       173
   macro avg       0.94      0.91      0.92       173
weighted avg       0.97      0.97      0.97       173

Optimized extra trees
              precision    recall  f1-score   support

         acc       0.93      1.00      0.97        42
        good       1.00      0.89      0.94         9
       unacc       1.00      0.99      1.00       114
       vgood       1.00      0.88      0.93         8

    accuracy                           0.98       173
   macro avg       0.98      0.94      0.96       173
weighted avg       0.98      0.98      0.98       173
```

Step 5: Compare all the models

In [20]:

```
from sklearn.metrics import accuracy_score

# accuracy of every model on the same test data
summary = pd.DataFrame({
    'model': ['Decision tree', 'Random forest (depth 6)', 'Extra trees (depth
6)',
              'Random forest (no limit)', 'Extra trees (no limit)'],
    'accuracy': [accuracy_score(label_test, dt_preds_test),
                 accuracy_score(label_test, rf_preds_test),
                 accuracy_score(label_test, et_preds_test),
                 accuracy_score(label_test, best_rf_preds),
                 accuracy_score(label_test, best_et_preds)]
})
summary['accuracy'] = summary['accuracy'].round(4)
summary
```

Out[20]:

	model	accuracy
0	Decision tree	0.9538
1	Random forest (depth 6)	0.8439
2	Extra trees (depth 6)	0.8150
3	Random forest (no limit)	0.9711
4	Extra trees (no limit)	0.9827

Conclusion

- The **decision tree** from Activity 4.01 already did well, with **95.4%** accuracy on the test data.
- With `max_depth=6`, the **random forest** (84.4%) and **extra trees** (81.5%) did worse. They missed all of the **good** cars because the trees were too shallow.
- After I removed the depth limit, both forests got much better. The random forest got **97.1%** and extra trees got **98.3%**, which is the best result. Extra trees also did the best on the rare **vgood** class.
- **persons** and **safety** are the most important features in every model. **doors** matters the least.

For the car rental company, I would use the **extra trees model with no depth limit**. It makes the fewest mistakes, and since many trees vote together, it is more reliable than a single tree. The company should look for cars that fit more people and have high safety ratings, because those are the cars clients are most likely to accept.